Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

Hands-On #3

Socket Raw in C: costruire un Ethernet Sniffer

Last Update: March 12, 2026

Scenario

Negli HO precedenti abbiamo lavorato con socket TCP e UDP: il kernel gestiva automaticamente incapsulamento, routing e consegna. Adesso scendiamo di livello!

Immaginiamo di dover sviluppare uno strumento di diagnostica di rete, oppure di dover capire esattamente cosa transita sul cavo *prima* che il kernel filtri i pacchetti. Con i socket ordinari non è possibile: il kernel consegna all'applicazione solo i byte del payload, dopo aver già rimosso tutti gli header.

Come si fa a vedere l'intero frame Ethernet, header incluso? Usando un *raw socket*.

Questa hands-on vi guiderà a costruire da zero un **sniffer Ethernet** in C: un programma che cattura ogni frame che transita su un'interfaccia di rete e ne decodifica gli header.

Concetti chiave da tenere a mente

Dove si collocano i raw socket nello stack

I socket ordinari (`SOCK_STREAM`, `SOCK_DGRAM`) operano al livello 4: il kernel aggiunge automaticamente gli header TCP/UDP, IP ed Ethernet, e all'arrivo li rimuove prima di consegnare i dati all'applicazione.

Con i *raw socket* è possibile scendere a **due livelli diversi**, a seconda di come si apre il socket:

```
Livello           Come si apre            Cosa si vede
L4 - Trasporto    SOCK_STREAM /           solo payload applicativo
                  SOCK_DGRAM              // HO precedenti
L3 - Rete         AF_INET + SOCK_RAW      header IP grezzo;
                                          Ethernet gestita dal
                                          kernel
L2 - Data-Link    AF_PACKET + SOCK_RAW    frame Ethernet completo
                                          // questo HO
L1 - Fisico       (non accessibile via    segnale elettrico /
                  socket)                 ottico
```

La distinzione pratica è importante:

- Con `AF_INET + SOCK_RAW` costruiamo o leggiamo l'header **IP** a mano, ma il kernel gestisce ancora il livello Ethernet per noi. È il livello usato per implementare `ping`, `traceroute` o uno scanner di porte — lo faremo nei prossimi HO.
- Con `AF_PACKET + SOCK_RAW` vediamo l'intero **frame Ethernet**, header MAC incluso, così come viaggia sul cavo. È il livello usato dagli sniffer (Wireshark scende a questo livello tramite `libpcap`).

La chiamata `socket()` per i raw socket a livello 2

Per aprire un raw socket che cattura tutti i protocolli a livello Ethernet:

```
int sd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
```

- `AF_PACKET` — famiglia per l'accesso al livello data-link (solo Linux, non POSIX standard).
- `SOCK_RAW` — il frame viene consegnato intero, header Ethernet incluso. Usare `SOCK_DGRAM` al posto di `SOCK_RAW` farebbe rimuovere al kernel l'header Ethernet prima della consegna.
- `htons(ETH_P_ALL)` — cattura *tutti* i protocolli (valore `0x0003` in network byte order). Si può sostituire con `htons(ETH_P_IP)` per ricevere solo frame IPv4.

La struttura del frame Ethernet 802.3

Ogni frame che transita sul cavo ha questa struttura:

6 byte	6 byte	2 byte	46-1500 byte	4 byte
MAC Dest	MAC Src	EtherType	Payload (IP/ARP/...)	FCS

Esempi di EtherType: 0x0800 = IPv4 0x0806 = ARP 0x86DD = IPv6

In C l'header Ethernet è rappresentato da `struct ethhdr` (in `<net/ethernet.h>`):

```
struct ethhdr {
    unsigned char h_dest[ETH_ALEN]; /* MAC destinazione (6 byte) */
    unsigned char h_source[ETH_ALEN]; /* MAC sorgente (6 byte)  */
    __be16 h_proto; /* EtherType, big-endian */
};
```

Il network byte order e `ntohs()`

Il campo `h_proto` arriva dalla rete in **big-endian**. Su architetture x86/x64 (little-endian), senza conversione leggeremmo `0x0008` invece di `0x0800` per IPv4. `ntohs()` esegue il byte-swap necessario.

Funzione	Direzione	Dimensione
`htons`	host → network	16 bit
`htonl`	host → network	32 bit
`ntohs`	network → host	16 bit
`ntohl`	network → host	32 bit

Privilegi necessari

I raw socket richiedono privilegi elevati. Senza di essi, `socket()` fallisce con `EPERM`.

```
sudo ./sniffer eth0
# oppure, una tantum:
sudo setcap cap_net_raw+ep ./sniffer
```

Codice di partenza

Il seguente sniffer funziona correttamente ma è incompleto: mancano la gestione degli errori, il filtraggio per interfaccia e la decodifica dell'EtherType. Analizza attentamente ogni sezione prima di procedere con gli esercizi.

Listing 1: sniffer_base.c — punto di partenza

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include <sys/socket.h>        /* socket(), bind(), AF_PACKET */
#include <netpacket/packet.h> /* struct sockaddr_ll */
#include <net/ethernet.h>      /* struct ethhdr, ETH_P_ALL */
#include <arpa/inet.h>         /* htons(), ntohs() */
#include <net/if.h>            /* if_nametoindex() */

#define BUFSIZE 65536

int main(int argc, char *argv[])
{
    /* PROBLEMA 1: nessun controllo sugli argomenti */
    const char *iface = argv[1];

    /* Apre il raw socket: cattura tutti i protocolli */
    int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    /* PROBLEMA 2: valore di ritorno di socket() non controllato */

    /* Recupera l'indice numerico dell'interfaccia */
    unsigned int ifindex = if_nametoindex(iface);

    /* Associa il socket all'interfaccia specificata */
    struct sockaddr_ll sll;
    memset(&sll, 0, sizeof(sll));
    sll.sll_family   = AF_PACKET;
    sll.sll_protocol = htons(ETH_P_ALL);
    sll.sll_ifindex  = ifindex;
    bind(sock, (struct sockaddr *)&sll, sizeof(sll));
    /* PROBLEMA 3: valore di ritorno di bind() non controllato */

    unsigned char buf[BUFSIZE];
    int frame = 0;

    while (1) {
        ssize_t n = recvfrom(sock, buf, BUFSIZE, 0, NULL, NULL);
        if (n < (ssize_t)sizeof(struct ethhdr)) continue;

        struct ethhdr *eth = (struct ethhdr *)buf;

        printf("Frame #%d  (%zd byte)\n", ++frame, n);

        /* Stampa i MAC address: ogni byte in esadecimale */
        printf("  DST: %02X:%02X:%02X:%02X:%02X:%02X\n",
            eth->h_dest[0],   eth->h_dest[1],   eth->h_dest[2],
            eth->h_dest[3],   eth->h_dest[4],   eth->h_dest[5]);
        printf("  SRC: %02X:%02X:%02X:%02X:%02X:%02X\n",
            eth->h_source[0], eth->h_source[1], eth->h_source[2],
            eth->h_source[3], eth->h_source[4], eth->h_source[5]);

        /* PROBLEMA 4: EtherType stampato senza conversione byte order
           e senza decodifica del protocollo */
        printf("  EtherType: 0x%04X\n", eth->h_proto);
```

```
    }

    close(sock);
    return 0;
}
```

Esercizi

Esercizio 1 — Identifica i problemi nel codice di partenza

1. Cosa succede se l'utente lancia `./sniffer` senza argomenti? Individua la riga che causa il crash e spiega perchè.
2. Perchè è importante controllare il valore di ritorno di `socket()` e di `bind()`? Quali errori potrebbero restituire e in quali circostanze?
3. Alla riga del `printf` dell'EtherType: perchè `eth->h_proto` stampato direttamente dà un valore sbagliato su x86? Qual è la correzione?
4. Cosa succede se si omette la `bind()`? Il programma continua a funzionare? Cambia il comportamento?

Esercizio 2 — Aggiungi la gestione degli errori

Partendo dal codice di partenza, aggiungi la gestione corretta di tutti gli errori indicati nei commenti `PROBLEMA`.

Listing 2: Schema da completare per la gestione errori

```c
/* Controlla argc e stampa un messaggio d'uso se errato */
if (argc != 2) { /* ... */ }

/* Controlla socket() */
int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if (sock < 0) { perror("socket"); exit(EXIT_FAILURE); }

/* Controlla if_nametoindex() */
unsigned int ifindex = if_nametoindex(iface);
if (ifindex == 0) { /* ... */ }

/* Controlla bind() */
if (bind(...) < 0) { /* ... */ }
```

▷ Suggerimento

Usa sempre `perror()` per stampare l'errore: legge automaticamente `errno` e aggiunge la stringa descrittiva. Ricorda di fare `close(sock)` prima di `exit()` ogni volta che il socket è già stato aperto.

[!] Attenzione

`if_nametoindex()` restituisce `0` in caso di errore (non `-1` come le altre syscall). Non confondere i due pattern.

Esercizio 3 — Decodifica l'EtherType

Scrivi una funzione che converte il valore numerico dell'EtherType in una stringa leggibile e usala nel loop di stampa.

Listing 3: Prototipo da implementare

```c
/*
 * Restituisce il nome del protocollo corrispondente all'
    EtherType.
 * 'proto' deve essere gia' in host byte order (dopo ntohs).
 * Esempio: ethertype_name(0x0800) -> "IPv4"
 */
const char *ethertype_name(uint16_t proto);
```

La funzione deve riconoscere almeno: `0x0800` (IPv4), `0x0806` (ARP), `0x86DD` (IPv6). Per tutti gli altri valori restituire la stringa `"sconosciuto"`.

Modifica il `printf` dell'EtherType in modo da stampare:

```
  EtherType: 0x0800 (IPv4)
```

▷ Suggerimento

Ricorda di applicare `ntohs(eth->h_proto)` *prima* di passare il valore a `ethertype_name()` e prima di confrontarlo con le costanti come `ETHERTYPE_IP`. Le costanti definite in `<net/ethernet.h>` sono già in host byte order.

Esercizio 4 — Statistiche e interruzione pulita

Aggiungi al programma un contatore per tipo di protocollo e un handler per `SIGINT` (Ctrl+C) che stampi le statistiche prima di uscire.

Listing 4: Struttura suggerita

```c
#include <signal.h>

/* Variabile globale: 1 = continua, 0 = termina */
volatile int running = 1;

/* Contatori globali */
unsigned long cnt_ipv4 = 0, cnt_arp = 0,
              cnt_ipv6 = 0, cnt_other = 0;

void handle_sigint(int sig)
{
    (void)sig;
    running = 0;    /* interrompe il loop */
}

int main(void)
{
    signal(SIGINT, handle_sigint);
    /* ... apertura socket e bind ... */

    while (running) {
        /* ... recvfrom e parsing ... */
        /* incrementa il contatore giusto in base all'EtherType
```

```
                */
    }

    /* Stampa le statistiche */
    printf("\n--- Statistiche ---\n");
    printf("IPv4 : %lu\n", cnt_ipv4);
    /* ... */
    close(sock);
    return 0;
}
```

Verifica che il programma stampi le statistiche quando premi `Ctrl+C` e che non lasci il socket aperto.

Esercizio 5 — Filtraggio da riga di comando

Estendi il programma in modo da accettare un secondo argomento opzionale che filtra per tipo di protocollo:

```
./sniffer eth0          # mostra tutti i frame
./sniffer eth0 arp      # mostra solo frame ARP
./sniffer eth0 ipv4     # mostra solo frame IPv4
./sniffer eth0 ipv6     # mostra solo frame IPv6
```

Se il secondo argomento non è presente, il programma mostra tutto (comportamento originale).

Domande di approfondimento

Rispondi a queste domande *dopo* aver completato gli esercizi.

Q1. Qual è la differenza tra `SOCK_RAW` e `SOCK_DGRAM` quando usati con `AF_PACKET`? In quale caso il kernel rimuove l'header Ethernet prima della consegna?

Q2. Senza la chiamata a `bind()`, il socket riceve frame da tutte le interfacce oppure solo da una? Fai una prova e descrivi il risultato.

Consegna: inviare zip con report e sorgenti al docente (`roberto.marino@unime.it`)
Prossimo passo: il tuo sniffer vede il frame Ethernet ma non ancora l'header IP che si nasconde nel payload. Questo problema verrà affrontato durante la prossima lezione...